Complete Source Code

```
import javax.swing.*;
import java.awt.*;
import java.io.File;
import java.nio.file.Files;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.*;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) {
        SQL.getInstance();
        new MainMenuWindow().open();
    }
}

class MainMenuWindow extends JFrame {

    public MainMenuWindow() {
        JPanel buttonPanel = new JPanel();
        buttonPanel.setLayout(new BoxLayout(buttonPanel, BoxLayout.Y_AXIS));

        buttonPanel.setAlignmentX(Component.CENTER_ALIGNMENT);
```

```
        buttonPanel.setAlignmentY(Component.CENTER_ALIGNMENT);
        buttonPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        JButton newStudent = new JButton("New Student");
        JButton listStudents = new JButton("List Students");
        JButton createTimetable = new JButton("Create Timetable");

        buttonPanel.add(newStudent);
        buttonPanel.add(Box.createVerticalStrut(10));
        buttonPanel.add(listStudents);
        buttonPanel.add(Box.createVerticalStrut(10));
        buttonPanel.add(createTimetable);

        getContentPane().add(buttonPanel, BorderLayout.CENTER);
        setSize(1280, 720);
        pack();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        newStudent.addActionListener(e -> new NewStudentMenu().open());
        listStudents.addActionListener(e -> new ListStudentsMenu().open());
        createTimetable.addActionListener(e -> new TimetableMenu().open());
    }

    public void open() {
        setVisible(true);
    }
}
```

```
class NewStudentMenu extends JFrame {

    private final JTextField studentID;
    private final JTextField fullName;
    private final ArrayList<JComboBox<String>> subjects;

    public NewStudentMenu() {
        StudentManager studentManager = StudentManager.getInstance();
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

        panel.setAlignmentX(Component.CENTER_ALIGNMENT);
        panel.setAlignmentY(Component.CENTER_ALIGNMENT);
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        studentID = new JTextField("Student ID");
        fullName = new JTextField("Full Name");
        subjects = new ArrayList<>();

        panel.add(fullName);
        panel.add(Box.createVerticalStrut(10));
        panel.add(studentID);
        panel.add(Box.createVerticalStrut(10));

        for (int i = 0; i < 6; i++) {
            JComboBox<String> subject = new JComboBox<>();
            for (Subject s : Subject.getAllSubjects()) {
                subject.addItem(s.getSubjectName());
```

```
            }
            subjects.add(subject);
            panel.add(Box.createVerticalStrut(4));
            panel.add(subject);
        }

        JButton addStudent = new JButton("Save");
        addStudent.addActionListener(e -> {
            ArrayList<Subject> studentSubjects = new ArrayList<>();
            for (JComboBox<String> subject : subjects) {
                studentSubjects.add(Subject.byName((String) subject.getSelectedItem()));
            }
            studentManager.addStudent(new Student(studentID.getText(), fullName.getText(),
studentSubjects));
            JOptionPane.showMessageDialog(null, "Student added successfully!");
            new Thread(() -> {
                try {
                    Connection connection = SQL.getInstance().getConnection();
                    String table = SQL.getInstance().getTable();
                    PreparedStatement statement = connection.prepareStatement("INSERT INTO " +
table + " (studentID, fullName, subjects) VALUES (?, ?, ?) ON DUPLICATE KEY UPDATE
fullName = ?, subjects = ?");
                    statement.setString(1, studentID.getText());
                    statement.setString(2, fullName.getText());
                    statement.setString(3, parseSubjects(studentSubjects));
                    statement.setString(4, fullName.getText());
                    statement.setString(5, parseSubjects(studentSubjects));
                    statement.executeUpdate();
                } catch (Exception ex) {
```

```
                throw new RuntimeException("Error saving student to database");
            }
        }).start();
    });

    JButton deleteStudent = new JButton("Delete");
    deleteStudent.addActionListener(e -> {
        Student student = studentManager.getStudent(studentID.getText());
        if (student != null) {
            studentManager.removeStudent(student);
            JOptionPane.showMessageDialog(null, "Student deleted successfully!");
            new Thread(() -> {
                try {
                    Connection connection = SQL.getInstance().getConnection();
                    String table = SQL.getInstance().getTable();
                    PreparedStatement statement = connection.prepareStatement("DELETE FROM "
+ table + " WHERE studentID = ?");
                    statement.setString(1, studentID.getText());
                    statement.executeUpdate();
                } catch (Exception ex) {
                    ex.printStackTrace();
                }
            }).start();
        } else {
            JOptionPane.showMessageDialog(null, "Student not found!");
        }
    });

    JButton back = new JButton("Back");
```

```
        back.addActionListener(e -> {
            new MainMenuWindow().open();
            dispose();
        });

        panel.add(Box.createVerticalStrut(10));
        panel.add(addStudent);
        panel.add(Box.createVerticalStrut(10));
        panel.add(deleteStudent);
        panel.add(Box.createVerticalStrut(10));
        panel.add(back);

        getContentPane().add(panel, BorderLayout.CENTER);
        setSize(1280, 720);
        pack();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void open() {
        setVisible(true);
    }

    private String parseSubjects(ArrayList<Subject> subjects) {
        StringBuilder sb = new StringBuilder();
        for (Subject s : subjects) {
            sb.append(s.getSubjectName()).append(",");
        }
        return sb.toString();
```

```
    }

    public void setFullName(String name) {
        fullName.setText(name);
    }

    public void setStudentID(String id) {
        studentID.setText(id);
    }

    public void setSubjects(ArrayList<Subject> subjects) {
        for (int i = 0; i < subjects.size(); i++) {
            this.subjects.get(i).setSelectedItem(subjects.get(i).getSubjectName());
        }
    }
}

class ListStudentsMenu extends JFrame {

    public ListStudentsMenu() {
        StudentManager studentManager = StudentManager.getInstance();
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

        panel.setAlignmentX(Component.CENTER_ALIGNMENT);
        panel.setAlignmentY(Component.CENTER_ALIGNMENT);
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

```
        JButton back = new JButton("Back");
        back.addActionListener(e -> {
            new MainMenuWindow().open();
            dispose();
        });

        for (Student student : studentManager.getStudents()) {
            JButton studentButton = new JButton(student.getFullName());
            studentButton.addActionListener(e -> {
                NewStudentMenu newStudentMenu = new NewStudentMenu();
                newStudentMenu.setFullName(student.getFullName());
                newStudentMenu.setStudentID(student.getStudentID());
                newStudentMenu.setSubjects(student.getSubjects());
                newStudentMenu.open();
                dispose();
            });
            panel.add(studentButton);
            panel.add(Box.createVerticalStrut(10));
        }

        panel.add(back);

        getContentPane().add(panel, BorderLayout.CENTER);
        setSize(1280, 720);
        pack();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
```

```
    public void open() {
        setVisible(true);
    }
}

class TimetableMenu extends JFrame {

    public TimetableMenu() {
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

        panel.setAlignmentX(Component.CENTER_ALIGNMENT);
        panel.setAlignmentY(Component.CENTER_ALIGNMENT);
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        JButton back = new JButton("Back");
        back.addActionListener(e -> {
            new MainMenuWindow().open();
            dispose();
        });

        List<String> timetable = buildTimetable();
        for (int i = 0; i < timetable.size(); i++) {
            JLabel day = new JLabel("Day " + (i + 1) + ": " + timetable.get(i));
            panel.add(day);
            panel.add(Box.createVerticalStrut(10));
        }
```

```
        panel.add(back);

        getContentPane().add(panel, BorderLayout.CENTER);
        setSize(1280, 720);
        pack();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void open() {
        setVisible(true);
    }

    private List<String> buildTimetable() {
        List<StringBuilder> timetable = new ArrayList<>();
        for (int i = 0; i < 5; i++) {
            timetable.add(new StringBuilder());
        }
        AtomicInteger currentDay = new AtomicInteger(1);
        AtomicInteger currentDayClasses = new AtomicInteger(0);
        AtomicInteger maxClassesPerDay = new AtomicInteger(0);
        for (Subject subject : Subject.getAllSubjects()) {
            maxClassesPerDay.getAndAdd(subject.getWeeklyClasses());
        }
        maxClassesPerDay.set(maxClassesPerDay.get() / 5);
        HashMap<Subject, Integer> subjects = new HashMap<>();
        for (Subject subject : Subject.getAllSubjects()) {
            subjects.put(subject, getSubjectPopularity(subject));
        }
```

```
subjects.entrySet().stream().sorted(Comparator.comparingInt(Map.Entry::getValue)).forEach(s -
> {
            if (currentDayClasses.get() >= maxClassesPerDay.get()) {
                currentDayClasses.set(0);
                currentDay.getAndIncrement();
            }
            for (int i = 0; i < s.getKey().getWeeklyClasses(); i++) {
                timetable.get(currentDay.get() - 1).append(s.getKey().getSubjectName()).append("; ");
            }
            currentDayClasses.set(currentDayClasses.get() + s.getKey().getWeeklyClasses());
        });
        List<String> result = new ArrayList<>();
        for (StringBuilder sb : timetable) {
            result.add(sb.toString());
        }
        return result;
    }

    private int getSubjectPopularity(Subject subject) {
        int popularity = 0;
        for (Student student : StudentManager.getInstance().getStudents()) {
            for (Subject s : student.getSubjects()) {
                if (s.getSubjectName().equals(subject.getSubjectName())) {
                    popularity++;
                }
            }
        }
        return popularity;
```

```
    }
}

class Student {
    private String studentID;
    private String fullName;
    private ArrayList<Subject> subjects;

    public Student(String id, String name, List<Subject> subjects) {
        this.studentID = id;
        this.fullName = name;
        this.subjects = new ArrayList<>(subjects);
    }

    public String getStudentID() {
        return studentID;
    }

    public String getFullName() {
        return fullName;
    }

    public ArrayList<Subject> getSubjects() {
        return subjects;
    }
}

class Subject {
```

```
private final String subjectName;
private final short groupNum;
private final int weeklyClasses;

public Subject(String subjectName, short groupNum, int sessionsPerWeek) {
    this.subjectName = subjectName;
    this.groupNum = groupNum;
    this.weeklyClasses = sessionsPerWeek;
}

public String getSubjectName() {
    return subjectName;
}

public short getGroupNum() {
    return groupNum;
}

public int getWeeklyClasses() {
    return weeklyClasses;
}

public static List<Subject> getAllSubjects() {
    File file = new File("subjects.txt");
    ArrayList<Subject> subjects = new ArrayList<>();
    try {
        for (String line : Files.readAllLines(file.toPath())) {
            String[] parts = line.split("-");
```

```
            int groupNum = Integer.parseInt(parts[0]);
            int weeklyClasses = Integer.parseInt(parts[2]);
            Subject subject = new Subject(parts[1], (short) groupNum, weeklyClasses);
            subjects.add(subject);
        }
    } catch (Exception e) {
        JDialog dialog = new JDialog();
        dialog.setLayout(new BorderLayout());
        dialog.add(new JLabel("Error reading subjects file"), BorderLayout.CENTER);
        dialog.setSize(200, 100);
        dialog.setVisible(true);
    }
    return subjects;
}

public static Subject byName(String name) {
    for (Subject s : getAllSubjects()) {
        if (s.getSubjectName().equals(name)) {
            return s;
        }
    }
    return null;
}
}

class StudentManager {
    private final ArrayList<Student> students;
    private static StudentManager instance;
```

```
private StudentManager() {
    students = new ArrayList<>();
}

public static StudentManager getInstance() {
    if (instance == null) {
        instance = new StudentManager();
    }
    return instance;
}

public List<Student> getStudents() {
    return Collections.unmodifiableList(students);
}

public void addStudent(Student student) {
    students.add(student);
}

public void removeStudent(Student student) {
    students.remove(student);
}

public Student getStudent(String studentID) {
    for (Student student : students) {
        if (student.getStudentID().equals(studentID)) {
            return student;
```

```
            }
        }
        return null;
    }
}

class SQL {

    private static SQL instance;
    private final Connection connection;
    private final String table;

    private SQL() {
        File file = new File("database.txt");
        if (!file.exists()) {
            try {
                file.createNewFile();
            } catch (Exception e) {
                throw new RuntimeException("Error creating database file");
            }
        }
        try {
            List<String> lines = Files.readAllLines(file.toPath());
            String url = lines.get(0);
            String username = lines.get(1);
            String password = lines.get(2);
            table = lines.get(3);
            Class.forName("com.mysql.jdbc.Driver");
```

```
            connection = DriverManager.getConnection(url, username, password);
            PreparedStatement statement = connection.prepareStatement("CREATE TABLE IF NOT
EXISTS `" + table + "` (`studentID` VARCHAR(255) NOT NULL , `fullName` VARCHAR(255)
NOT NULL , `subjects` VARCHAR(255) NOT NULL , PRIMARY KEY (`studentID`))");
            statement.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            throw new RuntimeException("Error reading database file");
        }
    }

    public static SQL getInstance() {
        if (instance == null) {
            instance = new SQL();
            retrieveStudents();
        }
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }

    public String getTable() {
        return table;
    }

    private static void retrieveStudents() {
        try {
```

```
            Connection connection = SQL.getInstance().getConnection();
            String table = SQL.getInstance().getTable();
            PreparedStatement statement = connection.prepareStatement("SELECT * FROM " +
table);
            ResultSet resultSet = statement.executeQuery();
            while (resultSet.next()) {
                String studentID = resultSet.getString("studentID");
                String fullName = resultSet.getString("fullName");
                String subjects = resultSet.getString("subjects");
                ArrayList<Subject> studentSubjects = new ArrayList<>();
                for (String subject : subjects.split(",")) {
                    studentSubjects.add(Subject.byName(subject));
                }
                StudentManager.getInstance().addStudent(new Student(studentID, fullName,
studentSubjects));
            }
        } catch (Exception e) {
            throw new RuntimeException("Error retrieving students from database");
        }
    }
}
```